DIGITAL ELECTRONICS PROJECT

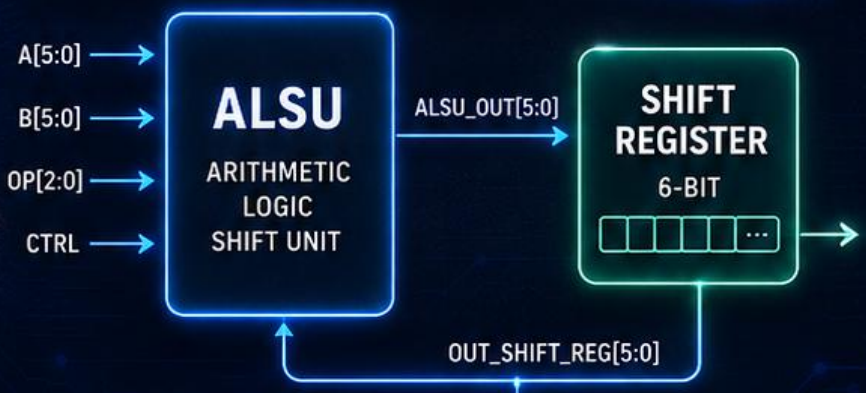
ALSU | SHIFT REGISTER

UVM VERIFICATION

DESIGN • VERIFICATION • IMPLEMENTATION

ALSU + SHIFT REGISTER

ACTIVE & PASSIVE UVM ENVIRONMENTS



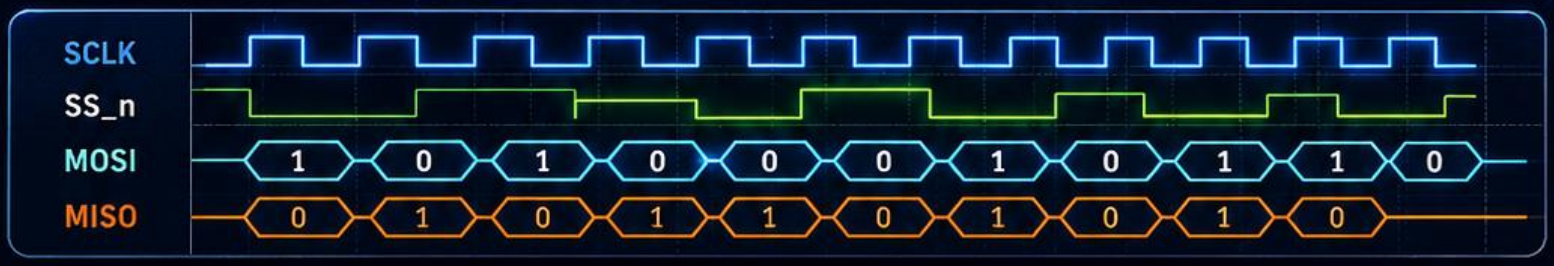
- ALSU + SHIFT REGISTER INTEGRATION
- ACTIVE / PASSIVE AGENTS
- SCOREBOARD & COVERAGE
- UVM METHODOLOGY
- MODULAR & SCALABLE ENVIRONMENT

ALSU ENVIRONMENT
(ACTIVE AGENT)

DRIVER → SEQUENCER → MONITOR

SHIFT REGISTER ENVIRONMENT
(PASSIVE AGENT)

MONITOR → SCOREBOARD → COVERAGE



Prepared by:
Mahmoud Abdelnasser Mahmoud

First :: shift_reg codes:

Design :

```
module shift_reg (clk, reset, serial_in, direction, mode, datain, dataout);
input clk, reset, serial_in, direction, mode;
input [5:0] datain;
output reg [5:0] dataout;

always @(posedge clk or posedge reset) begin
   if (reset)
      dataout <= 0;
   else
      if (mode) // rotate
         if (direction) // left
            dataout <= {datain[4:0], datain[5]};
         else
            dataout <= {datain[0], datain[5:1]};
      else // shift
         if (direction) // left
            dataout <= {datain[4:0], serial_in};
         else
            dataout <= {serial_in, datain[5:1]};
end
endmodule
```

interface :

```
interface shift_reg_if (clk);
  input clk;
  logic reset;
  logic serial_in, direction, mode;
  logic [5:0] datain, dataout;
endinterface : shift_reg_if
```

shared package :

```
package  shared_package_shift_reg;
typedef enum  {SHIFT , ROTATE  }  mode_e;
typedef enum {RIGHT , LEFT} direction_e;

endpackage
```

agent :

```
package shift_reg_agent_pkg;
```

```
import shift_reg_monitor_pkg::*;
import shift_reg_sequencer_pkg::*;
import shift_reg_driver_pkg::*;
import shift_reg_cfg_pkg::*;
import uvm_pkg::*;
import shift_reg_seq_item_pkg::*;
`include "uvm_macros.svh"

class shift_reg_agent extends uvm_agent;

  `uvm_component_utils(shift_reg_agent)


  shift_reg_monitor   mon;
  shift_reg_sequencer sqr;
  shift_reg_driver    drv;


  shift_reg_config shift_reg_cfg;


  uvm_analysis_port #(shift_reg_seq_item) agt_ap;


  function new (string name = "shift_reg_agent", uvm_component parent = null);
    super.new(name, parent);
  endfunction


  function void build_phase(uvm_phase phase);
    super.build_phase(phase);


    if (!uvm_config_db #(shift_reg_config)::get(this , "" , "SHIFT_CFG" , shift_reg_cfg))
begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end


    if (shift_reg_cfg.is_active == UVM_ACTIVE) begin
      sqr = shift_reg_sequencer::type_id::create("sqr", this);
      drv = shift_reg_driver::type_id::create("drv", this);
    end


    mon = shift_reg_monitor::type_id::create("mon", this);


    agt_ap = new("agt_ap", this);

  endfunction
```

```
  function void connect_phase(uvm_phase phase);


    mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;


    if (shift_reg_cfg.is_active == UVM_ACTIVE) begin
      drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
      drv.seq_item_port.connect(sqr.seq_item_export);
    end


    mon.mon_ap.connect(agt_ap);

  endfunction

endclass

endpackage
```

config :

```
package shift_reg_cfg_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class shift_reg_config extends uvm_object;
`uvm_object_utils(shift_reg_config)

  virtual shift_reg_if shift_reg_vif;
   uvm_active_passive_enum is_active ;
  function new(string name = "shift_reg_config");
    super.new(name);
  endfunction
endclass
endpackage
```

driver :

```
package shift_reg_driver_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

import shift_reg_cfg_pkg::*;
import shift_reg_seq_item_pkg::*;

class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);

  `uvm_component_utils(shift_reg_driver)
```

```
  shift_reg_seq_item stim_seq_item;
  virtual shift_reg_if shift_reg_vif;
  shift_reg_config shift_reg_cfg;

  function new(string name = "shift_reg_driver", uvm_component parent = null);
    super.new(name, parent);
  endfunction


  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    if(!uvm_config_db #(shift_reg_config)::get(this, "", "SHIFT_CFG", shift_reg_cfg))
begin
      `uvm_fatal("build_phase", "Unable to get shift_reg config")
    end
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);

    shift_reg_vif = shift_reg_cfg.shift_reg_vif;
  endfunction


  task run_phase(uvm_phase phase);
    super.run_phase(phase);

    forever begin

      seq_item_port.get_next_item(stim_seq_item);

      // drive signals
      shift_reg_vif.direction = stim_seq_item.direction;
      shift_reg_vif.mode      = stim_seq_item.mode;
      shift_reg_vif.datain    = stim_seq_item.datain;
      shift_reg_vif.serial_in = stim_seq_item.serial_in;



      #2;

      seq_item_port.item_done();

      `uvm_info("DRIVER",
        stim_seq_item.convert2string_stimulus(),
        UVM_HIGH)

    end
  endtask

endclass
```

```
endpackage
```

environment :

```
package shift_reg_env_pkg;
import shift_reg_agent_pkg ::*;
import shift_reg_coverage_pkg ::*;

import uvm_pkg::*;
`include "uvm_macros.svh"

import shift_reg_driver_pkg::*;

class shift_reg_env extends uvm_env;

  `uvm_component_utils(shift_reg_env)
  shift_reg_agent agt;
  shift_reg_coverage cov ;
  function new(string name = "shift_reg_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  agt = shift_reg_agent::type_id::create("agt", this);
  cov = shift_reg_coverage::type_id::create("cov", this);
endfunction: build_phase

function void connect_phase(uvm_phase phase);
  agt.agt_ap.connect(cov.cov_export);
endfunction
endclass
endpackage
```

main seq:

```
package shift_reg_main_sequence_pkg;
import shared_package_shift_reg ::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
import shift_reg_seq_item_pkg ::*;
class shift_reg_main_sequence extends uvm_sequence #(shift_reg_seq_item);
  `uvm_object_utils(shift_reg_main_sequence);
  shift_reg_seq_item seq_item;
```

```
  function new (string name = "shift_reg_main_sequence");
    super.new(name);
  endfunction

  task body;
    repeat(10000) begin
      seq_item = shift_reg_seq_item::type_id::create("seq_item");
      start_item(seq_item);
      assert(seq_item.randomize());
      finish_item(seq_item);
    end
  endtask
endclass


class shift_reg_reset_sequence extends uvm_sequence #(shift_reg_seq_item);
  `uvm_object_utils(shift_reg_reset_sequence);
  shift_reg_seq_item seq_item;

  function new (string name = "shift_reg_reset_sequence");
    super.new(name);
  endfunction

  task body;
    seq_item = shift_reg_seq_item::type_id::create("seq_item");
    start_item(seq_item);
    seq_item.reset = 1;
    seq_item.serial_in = 0;
    seq_item.direction = direction_e'(0);
    seq_item.mode = mode_e'(0);
    seq_item.datain = 0;
    finish_item(seq_item);
  endtask
endclass
endpackage
```

monitor :

```
package shift_reg_monitor_pkg;

import uvm_pkg::*;
`include "uvm_macros.svh"

import shift_reg_seq_item_pkg::*;
import shared_package_shift_reg::*;

class shift_reg_monitor extends uvm_monitor;

  `uvm_component_utils(shift_reg_monitor)

  virtual shift_reg_if shift_reg_vif;
  shift_reg_seq_item rsp_seq_item;
```

```
  uvm_analysis_port #(shift_reg_seq_item) mon_ap;

  function new(string name = "shift_reg_monitor", uvm_component parent = null);
    super.new(name, parent);
  endfunction


  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction


  task run_phase(uvm_phase phase);
    super.run_phase(phase);

    forever begin

      rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");

      #2;
      rsp_seq_item.direction = direction_e'(shift_reg_vif.direction);
      rsp_seq_item.mode      = mode_e'(shift_reg_vif.mode);
      rsp_seq_item.datain    = shift_reg_vif.datain;
      rsp_seq_item.serial_in = shift_reg_vif.serial_in;
      rsp_seq_item.dataout   = shift_reg_vif.dataout;




      mon_ap.write(rsp_seq_item);

      `uvm_info("MONITOR",
        rsp_seq_item.convert2string(),
        UVM_HIGH)

    end
  endtask

endclass

endpackage
```

seq item :

```
package shift_reg_seq_item_pkg;
`include "uvm_macros.svh"
import uvm_pkg::*;
import shared_package_shift_reg::*;

class shift_reg_seq_item extends uvm_sequence_item;
```

```
  `uvm_object_utils(shift_reg_seq_item)

  rand bit reset;
  rand bit serial_in;
  rand direction_e direction;
  rand mode_e mode;
  rand bit [5:0] datain;
  logic [5:0] dataout;

  function new(string name = "shift_reg_seq_item");
    super.new(name);
  endfunction

  function string convert2string();
    return $sformatf("%s reset = 0b%b, serial_in = 0b%b, direction = %s, mode = %s,
datain = 0b%h, dataout = 0b%h",
                  super.convert2string()  , reset, serial_in, direction, mode, datain,
dataout);
  endfunction

  function string convert2string_stimulus();
    return $sformatf("reset = 0b%b, serial_in = 0b%b, direction = %s, mode = %s, datain =
0b%h",
                     reset, serial_in, direction, mode, datain);
  endfunction

  constraint rst_constrain {
    reset dist {1 := 10, 0 := 90};
  }

endclass
endpackage
```

sequencer :

```
package shift_reg_sequencer_pkg;
import shift_reg_seq_item_pkg ::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
class shift_reg_sequencer extends uvm_sequencer #(shift_reg_seq_item);
  `uvm_component_utils(shift_reg_sequencer);

  function new (string name = "shift_reg_sequencer", uvm_component parent = null);
    super.new(name, parent);
  endfunction

endclass
endpackage
```

second :: ALSU codes :

design :

```
module ALSU(ALSU_if.DUT alsu , input logic signed [5:0] out_shift_reg);

parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER = "ON";

reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg;
reg direction_reg, serial_in_reg;
reg cin_reg;
reg [2:0] opcode_reg;
reg signed [2:0] A_reg, B_reg;

wire invalid_red_op, invalid_opcode, invalid;


assign invalid_red_op = (alsu.red_op_A | alsu.red_op_B) & (alsu.opcode[1] |
alsu.opcode[2]);
assign invalid_opcode = alsu.opcode[1] & alsu.opcode[2];
assign invalid = invalid_red_op | invalid_opcode;


always @(posedge alsu.clk or posedge alsu.rst) begin
  if(alsu.rst) begin
     cin_reg <= 0;
     red_op_B_reg <= 0;
     red_op_A_reg <= 0;
     bypass_B_reg <= 0;
     bypass_A_reg <= 0;
     direction_reg <= 0;
     serial_in_reg <= 0;
     opcode_reg <= 0;
     A_reg <= 0;
     B_reg <= 0;
  end else begin
     cin_reg <= alsu.cin;
     red_op_B_reg <= alsu.red_op_B;
     red_op_A_reg <= alsu.red_op_A;
     bypass_B_reg <= alsu.bypass_B;
     bypass_A_reg <= alsu.bypass_A;
     direction_reg <= alsu.direction;
     serial_in_reg <= alsu.serial_in;
     opcode_reg <= alsu.opcode;
     A_reg <= alsu.A;
     B_reg <= alsu.B;
  end
end

always @(posedge alsu.clk or posedge alsu.rst) begin
  if(alsu.rst) begin
```

```
    alsu.leds <= 0;
  end else begin
    if (invalid)
      alsu.leds <= ~alsu.leds;
    else
      alsu.leds <= 0;
  end
end

always @(posedge alsu.clk or posedge alsu.rst) begin
  if(alsu.rst) begin
    alsu.out <= 0;
  end
  else begin

    if (bypass_A_reg && bypass_B_reg)
      alsu.out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;

    else if (bypass_A_reg)
      alsu.out <= A_reg;

    else if (bypass_B_reg)
      alsu.out <= B_reg;

    else if (invalid)
      alsu.out <= 0;

    else begin
      case (opcode_reg)


        3'h0: begin
          if (red_op_A_reg && red_op_B_reg)
            alsu.out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
          else if (red_op_A_reg)
            alsu.out <= |A_reg;
          else if (red_op_B_reg)
            alsu.out <= |B_reg;
          else
            alsu.out <= A_reg | B_reg;
        end


        3'h1: begin
          if (red_op_A_reg && red_op_B_reg)
            alsu.out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
          else if (red_op_A_reg)
            alsu.out <= ^A_reg;
          else if (red_op_B_reg)
            alsu.out <= ^B_reg;
          else
            alsu.out <= A_reg ^ B_reg;
        end
```

```
        3'h2: begin
          if (FULL_ADDER == "ON")
            alsu.out <= A_reg + B_reg + cin_reg;
          else
            alsu.out <= A_reg + B_reg;
        end


        3'h3: alsu.out <= A_reg * B_reg;


        3'h4: alsu.out <= out_shift_reg;


        3'h5: alsu.out <= out_shift_reg;

      endcase
    end
  end
end

endmodule
```

interface :

```
interface ALSU_if (clk);
  input clk ;
  logic cin;
  logic rst;
  logic red_op_A;
  logic red_op_B;
  logic bypass_A;
  logic bypass_B;
  logic direction;
  logic serial_in;

  logic [2:0] opcode;
  logic signed [2:0] A;
  logic signed [2:0] B;

  logic [15:0] leds;
  logic signed [5:0] out;

  // Modport for DUT
  modport DUT (
    input  clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in,
opcode, A, B,
    output leds, out
  );
```

```
endinterface
```

shared package :

```
package  shared_package_ALSU;

typedef enum  {RIGHT , LEFT} direction_e;
typedef enum bit [2:0] {OR=0, XOR=1, ADD=2, MULT=3, SHIFT=4, ROTATE=5, INVALID_6=6,
INVALID_7=7} opcode_e;
typedef enum {MAXPOS=3, ZERO=0, MAXNEG=-4} corner_case_e;
typedef enum bit [2:0] {first = 3'b001, second = 3'b010, third = 3'b100} one_bit_high_e;


endpackage
```

agent :

```
package ALSU_agent_pkg;
import ALSU_monitor_pkg ::*;
import ALSU_sequencer_pkg ::*;
import ALSU_driver_pkg ::*;
import ALSU_cfg_pkg ::*;
import uvm_pkg ::*;
import ALSU_seq_item_pkg ::*;
`include "uvm_macros.svh"
class ALSU_agent extends uvm_agent;
  `uvm_component_utils(ALSU_agent)
  ALSU_monitor mon ;
  ALSU_sequencer sqr;
  ALSU_driver drv;
  ALSU_config ALSU_cfg;
  uvm_analysis_port #(ALSU_seq_item) agt_ap;

  function new (string name = "ALSU_agent", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(ALSU_config)::get(this , "" , "CFG" , ALSU_cfg)) begin
      `uvm_fatal("build_phase" , "Unable to get configuration object")
    end

    sqr = ALSU_sequencer::type_id::create("sqr",this);
    drv = ALSU_driver::type_id::create("drv",this);
    agt_ap = new("agt_ap", this);
    mon = ALSU_monitor::type_id::create("mon" , this ) ;
  endfunction


  function void connect_phase(uvm_phase phase);
  drv.ALSU_vif = ALSU_cfg.ALSU_vif;
```

```
  mon.ALSU_vif = ALSU_cfg.ALSU_vif;
  drv.seq_item_port.connect(sqr.seq_item_export);
  mon.mon_ap.connect(agt_ap);
endfunction

endclass
endpackage
```

config :

```
package ALSU_cfg_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class ALSU_config extends uvm_object;
`uvm_object_utils(ALSU_config)

  virtual ALSU_if ALSU_vif;

  function new(string name = "ALSU_config");
    super.new(name);
  endfunction
endclass
endpackage
```

coverage collector :

```
package ALSU_coverage_pkg;

  import ALSU_seq_item_pkg::*;
  import uvm_pkg::*;
  import shared_package_ALSU::*;
  `include "uvm_macros.svh"

  class ALSU_coverage extends uvm_component;
    `uvm_component_utils(ALSU_coverage)

    uvm_analysis_export #(ALSU_seq_item) cov_export;
    uvm_tlm_analysis_fifo #(ALSU_seq_item) cov_fifo;
    ALSU_seq_item seq_item_cov;


    logic signed [2:0] A_cov, B_cov;
    opcode_e opcode_cov;
    logic cin_cov, red_op_A_cov, red_op_B_cov;
    logic bypass_A_cov, bypass_B_cov;
    logic direction_cov, serial_in_cov;


    covergroup cvr_grp;
```

```
A_cp : coverpoint A_cov {
  bins A_data_0       = {0};
  bins A_data_max     = {MAXPOS};
  bins A_data_min     = {MAXNEG};
  bins A_data_default = default;
}


B_cp : coverpoint B_cov {
  bins B_data_0       = {0};
  bins B_data_max     = {MAXPOS};
  bins B_data_min     = {MAXNEG};
  bins B_data_default = default;
}


ALU_cp : coverpoint opcode_cov {
  bins bitwise[] = {OR, XOR};
  bins arith[]   = {ADD, MULT};
  bins shift[]   = {SHIFT, ROTATE};


  ignore_bins invalid = {INVALID_6, INVALID_7};
}



cross A_cov, B_cov, opcode_cov {
  bins arith = binsof(opcode_cov) intersect {ADD, MULT};
}

cross cin_cov, opcode_cov {
  bins add = binsof(opcode_cov) intersect {ADD};
}

cross direction_cov, opcode_cov {
  bins shift_rot = binsof(opcode_cov) intersect {SHIFT, ROTATE};
}

cross serial_in_cov, opcode_cov {
  bins shift = binsof(opcode_cov) intersect {SHIFT};
}

cross A_cov, B_cov, opcode_cov {
  bins bitwise = binsof(opcode_cov) intersect {OR, XOR};
  bins A_walk  = binsof(A_cov) intersect {3'b001,3'b010,3'b100};
  bins B_zero  = binsof(B_cov) intersect {0};
}

cross B_cov, A_cov, opcode_cov {
  bins bitwise = binsof(opcode_cov) intersect {OR, XOR};
```

```
    bins B_walk  = binsof(B_cov) intersect {3'b001,3'b010,3'b100};
    bins A_zero  = binsof(A_cov) intersect {0};
  }


  cross opcode_cov, red_op_A_cov, red_op_B_cov {


    ignore_bins red_invalid =
      binsof(opcode_cov) intersect {ADD, MULT, SHIFT, ROTATE} &&
      (binsof(red_op_A_cov) intersect {1} ||
       binsof(red_op_B_cov) intersect {1});


    bins bitwise_red =
      binsof(opcode_cov) intersect {OR, XOR};

  }

endgroup


function new(string name = "ALSU_coverage", uvm_component parent = null);
  super.new(name, parent);
  cvr_grp = new();
endfunction


function void build_phase(uvm_phase phase);
  super.build_phase(phase);
  cov_export = new("cov_export", this);
  cov_fifo   = new("cov_fifo", this);
endfunction


function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);
  cov_export.connect(cov_fifo.analysis_export);
endfunction

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    cov_fifo.get(seq_item_cov);


    A_cov           = seq_item_cov.A;
    B_cov           = seq_item_cov.B;
    opcode_cov      = seq_item_cov.opcode;
    cin_cov         = seq_item_cov.cin;
    red_op_A_cov    = seq_item_cov.red_op_A;
    red_op_B_cov    = seq_item_cov.red_op_B;
    bypass_A_cov    = seq_item_cov.bypass_A;
```

```
        bypass_B_cov    = seq_item_cov.bypass_B;
        direction_cov   = seq_item_cov.direction;
        serial_in_cov   = seq_item_cov.serial_in;


        cvr_grp.sample();
      end
    endtask

  endclass

endpackage
```

driver :

```
package ALSU_driver_pkg;
import uvm_pkg::*;
`include "uvm_macros.svh"
import ALSU_cfg_pkg::*;
import shared_package_ALSU ::*;
 import ALSU_seq_item_pkg ::*;
class ALSU_driver extends uvm_driver #(ALSU_seq_item);
`uvm_component_utils (ALSU_driver)
ALSU_seq_item stim_seq_item ;
virtual ALSU_if ALSU_vif ;
ALSU_config ALSU_cfg ;
function new (string name = "ALSU_driver" ,uvm_component parent= null) ;
super.new(name , parent ) ;
endfunction

function void build_phase (uvm_phase phase ) ;
super.build_phase(phase ) ;
if(!uvm_config_db #(ALSU_config):: get(this, "" ,"CFG",ALSU_cfg)) begin
`uvm_fatal("build_phase" , "unable to get config")
end
endfunction
function void connect_phase (uvm_phase phase);
super.connect_phase (phase);

ALSU_vif = ALSU_cfg.ALSU_vif ;
endfunction
task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    stim_seq_item = ALSU_seq_item::type_id::create("stim_seq_item");
    seq_item_port.get_next_item(stim_seq_item);

    ALSU_vif.direction = direction_e'(stim_seq_item.direction);
    ALSU_vif.cin       = stim_seq_item.cin;
    ALSU_vif.red_op_A  = stim_seq_item.red_op_A;
    ALSU_vif.red_op_B  = stim_seq_item.red_op_B;
    ALSU_vif.bypass_A  = stim_seq_item.bypass_A;
```

```
    ALSU_vif.bypass_B  = stim_seq_item.bypass_B;
    ALSU_vif.opcode    = opcode_e'(stim_seq_item.opcode);
    ALSU_vif.serial_in = stim_seq_item.serial_in;
    ALSU_vif.A         = stim_seq_item.A;
    ALSU_vif.B         = stim_seq_item.B;
    ALSU_vif.rst       = stim_seq_item.rst;

    @(negedge ALSU_vif.clk);
    seq_item_port.item_done();

    `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
  end
endtask
endclass
endpackage
```

environment:

```
package ALSU_env_pkg;

import ALSU_agent_pkg::*;
import ALSU_coverage_pkg::*;
import ALSU_scoreboard_pkg::*;
import shift_reg_agent_pkg::*;

import uvm_pkg::*;
`include "uvm_macros.svh"

class ALSU_env extends uvm_env;

  `uvm_component_utils(ALSU_env)


  ALSU_agent agt;
  shift_reg_agent shift_agt;


  ALSU_scoreboard sb;
  ALSU_coverage cov;

  function new(string name = "ALSU_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction


  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    agt       = ALSU_agent::type_id::create("agt", this);
    shift_agt = shift_reg_agent::type_id::create("shift_agt", this);
    sb  = ALSU_scoreboard::type_id::create("sb", this);
```

```
    cov = ALSU_coverage::type_id::create("cov", this);

  endfunction
  function void connect_phase(uvm_phase phase);

    agt.agt_ap.connect(sb.sb_export);
    agt.agt_ap.connect(cov.cov_export);



  endfunction

endclass

endpackage
```

main seq:

```
package ALSU_main_sequence_pkg;
import shared_package_ALSU ::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
import ALSU_seq_item_pkg ::*;
class ALSU_main_sequence extends uvm_sequence #(ALSU_seq_item);
  `uvm_object_utils(ALSU_main_sequence);
  ALSU_seq_item seq_item;

  function new (string name = "ALSU_main_sequence");
    super.new(name);
  endfunction

  task body;
    repeat(10000) begin
      seq_item = ALSU_seq_item::type_id::create("seq_item");
      start_item(seq_item);
      assert(seq_item.randomize());
      finish_item(seq_item);
    end
  endtask
endclass


class ALSU_reset_sequence extends uvm_sequence #(ALSU_seq_item);
  `uvm_object_utils(ALSU_reset_sequence);
  ALSU_seq_item seq_item;

  function new (string name = "ALSU_reset_sequence");
    super.new(name);
  endfunction

  task body;
  seq_item = ALSU_seq_item::type_id::create("seq_item");
```

```
  start_item(seq_item);

  seq_item.cin       = 0;
  seq_item.red_op_A  = 0;
  seq_item.red_op_B  = 0;
  seq_item.bypass_A  = 0;
  seq_item.bypass_B  = 0;
  seq_item.direction = direction_e'(0);
  seq_item.serial_in = 0;
  seq_item.opcode    = opcode_e'(0);
  seq_item.A         = 0;
  seq_item.B         = 0;
  seq_item.rst       = 1;

  finish_item(seq_item);
endtask
endclass
endpackage
```

monitor :

```
package ALSU_monitor_pkg;
`include "uvm_macros.svh"
import shared_package_ALSU ::*;
import ALSU_seq_item_pkg::*;
  import uvm_pkg::*;

class ALSU_monitor extends uvm_monitor;
  `uvm_component_utils(ALSU_monitor)

   uvm_analysis_port #(ALSU_seq_item) mon_ap;
  virtual ALSU_if ALSU_vif;
  ALSU_seq_item rsp_seq_item;

  function new(string name = "ALSU_monitor", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    mon_ap = new("mon_ap", this);
  endfunction: build_phase

task run_phase(uvm_phase phase);
  super.run_phase(phase);
  forever begin
    rsp_seq_item = ALSU_seq_item::type_id::create("rsp_seq_item");

    @(negedge ALSU_vif.clk);

    rsp_seq_item.direction = direction_e'(ALSU_vif.direction);
    rsp_seq_item.cin       = ALSU_vif.cin;
```

```
    rsp_seq_item.red_op_A  = ALSU_vif.red_op_A;
    rsp_seq_item.red_op_B  = ALSU_vif.red_op_B;
    rsp_seq_item.bypass_A  = ALSU_vif.bypass_A;
    rsp_seq_item.bypass_B  = ALSU_vif.bypass_B;
    rsp_seq_item.opcode    = opcode_e'(ALSU_vif.opcode);
    rsp_seq_item.serial_in = ALSU_vif.serial_in;
    rsp_seq_item.A         = ALSU_vif.A;
    rsp_seq_item.B         = ALSU_vif.B;
    rsp_seq_item.rst       = ALSU_vif.rst;
    rsp_seq_item.out       = ALSU_vif.out;
    rsp_seq_item.leds      = ALSU_vif.leds;

    mon_ap.write(rsp_seq_item);

    `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
  end
endtask

endclass
endpackage
```

scoreboard :

```
package ALSU_scoreboard_pkg;

parameter INPUT_PRIORITY = "A";
parameter FULL_ADDER     = "ON";

import shared_package_ALSU::*;
import ALSU_seq_item_pkg::*;
import shift_reg_seq_item_pkg::*;
import uvm_pkg::*;
`include "uvm_macros.svh"

class ALSU_scoreboard extends uvm_scoreboard;

  `uvm_component_utils(ALSU_scoreboard)


  uvm_analysis_export #(ALSU_seq_item) sb_export;
  uvm_tlm_analysis_fifo #(ALSU_seq_item) sb_fifo;
  ALSU_seq_item seq_item_sb;


  uvm_analysis_export #(shift_reg_seq_item) shift_export;
  uvm_tlm_analysis_fifo #(shift_reg_seq_item) shift_fifo;
  shift_reg_seq_item shift_item;


  int error_count = 0;
  int correct_count = 0;
```

```
bit [15:0] leds_ref;
bit signed [5:0] out_ref;

function new(string name = "ALSU_scoreboard", uvm_component parent = null);
  super.new(name, parent);
endfunction


function void build_phase(uvm_phase phase);
  super.build_phase(phase);

  sb_export    = new("sb_export", this);
  sb_fifo      = new("sb_fifo", this);

  shift_export = new("shift_export", this);
  shift_fifo   = new("shift_fifo", this);
endfunction


function void connect_phase(uvm_phase phase);
  super.connect_phase(phase);

  sb_export.connect(sb_fifo.analysis_export);
  shift_export.connect(shift_fifo.analysis_export);
endfunction


task run_phase(uvm_phase phase);
  super.run_phase(phase);

  forever begin

    sb_fifo.get(seq_item_sb);
    shift_fifo.get(shift_item);

    ref_model(seq_item_sb);


    if (seq_item_sb.opcode == 3'h4 || seq_item_sb.opcode == 3'h5) begin

      if (seq_item_sb.out !== shift_item.dataout) begin
        `uvm_error("run_phase",
          $sformatf("SHIFT/ROTATE ERROR | DUT=%0h SHIFT=%0h",
          seq_item_sb.out, shift_item.dataout))
        error_count++;
      end
      else begin
        correct_count++;
      end

    end
```

```
    else begin

      if (seq_item_sb.out !== out_ref || seq_item_sb.leds !== leds_ref) begin
        `uvm_error("run_phase",
         $sformatf("Mismatch | DUT=%0h REF=%0h",
         seq_item_sb.out, out_ref))
        error_count++;
      end
      else begin
        correct_count++;
      end

    end

  end
endtask


task ref_model(ALSU_seq_item item);

  static bit signed [2:0] A_reg = 0, B_reg = 0;
  static bit [2:0] opcode_reg = 0;
  static bit cin_reg = 0;
  static bit red_op_A_reg = 0, red_op_B_reg = 0;
  static bit bypass_A_reg = 0, bypass_B_reg = 0;

  static bit signed [5:0] out_reg = 0;
  static bit [15:0] leds_reg = 0;

  bit invalid;


  if (item.rst) begin
    A_reg = 0;
    B_reg = 0;
    opcode_reg = 0;
    cin_reg = 0;
    red_op_A_reg = 0;
    red_op_B_reg = 0;
    bypass_A_reg = 0;
    bypass_B_reg = 0;

    out_reg  = 0;
    leds_reg = 0;
  end

  else begin


    invalid = (item.opcode[1] & item.opcode[2]);


    if (invalid)
```

```
      leds_reg = ~leds_reg;
    else
      leds_reg = 0;


    if (bypass_A_reg && bypass_B_reg)
      out_reg = A_reg;

    else if (bypass_A_reg)
      out_reg = A_reg;

    else if (bypass_B_reg)
      out_reg = B_reg;

    else if (invalid)
      out_reg = 0;

    else begin
      case (opcode_reg)

        // OR
        3'h0: begin
          if (red_op_A_reg)
            out_reg = |A_reg;
          else if (red_op_B_reg)
            out_reg = |B_reg;
          else
            out_reg = A_reg | B_reg;
        end

        // XOR
        3'h1: begin
          if (red_op_A_reg)
            out_reg = ^A_reg;
          else if (red_op_B_reg)
            out_reg = ^B_reg;
          else
            out_reg = A_reg ^ B_reg;
        end

        // ADD
        3'h2: begin
          if (FULL_ADDER == "ON")
            out_reg = A_reg + B_reg + cin_reg;
          else
            out_reg = A_reg + B_reg;
        end

        // MULT
        3'h3: begin
          out_reg = A_reg * B_reg;
        end
```

```
          3'h4: out_reg = out_reg;
          3'h5: out_reg = out_reg;

          default: out_reg = 0;

        endcase
      end


      A_reg = item.A;
      B_reg = item.B;
      opcode_reg = item.opcode;
      cin_reg = item.cin;
      red_op_A_reg = item.red_op_A;
      red_op_B_reg = item.red_op_B;
      bypass_A_reg = item.bypass_A;
      bypass_B_reg = item.bypass_B;

    end


    out_ref  = out_reg;
    leds_ref = leds_reg;

  endtask

endclass

endpackage
```

seq item :

```
package ALSU_seq_item_pkg;

`include "uvm_macros.svh"
import uvm_pkg::*;
import shared_package_ALSU::*;

class ALSU_seq_item extends uvm_sequence_item;
  `uvm_object_utils(ALSU_seq_item)

  // Signals
  rand bit clk, rst, cin, red_op_A, red_op_B, bypass_B, bypass_A, serial_in;
  rand direction_e direction;
  rand bit signed [2:0] A, B;
  rand opcode_e opcode;

  logic [5:0] out;
  logic [15:0] leds;

  // Extra randoms
```

```
  rand corner_case_e corner_case;
  rand bit signed [2:0] other;
  rand one_bit_high_e one_bit_high;
  rand opcode_e arr_opcode[6];

  function new(string name = "ALSU_seq_item");
    super.new(name);
  endfunction


  function string convert2string();
    return $sformatf("%s reset = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b,
bypass_A = 0b%b, bypass_B = 0b%b, serial_in = 0b%b, direction = %s, opcode = %s, A =
0b%b, B = 0b%b, leds = 0b%b, out = 0b%b",
                      super.convert2string(), rst, cin, red_op_A, red_op_B, bypass_A,
bypass_B, serial_in, direction, opcode, A, B, leds, out);
  endfunction

  function string convert2string_stimulus();
    return $sformatf("%s reset = 0b%b, cin = 0b%b, red_op_A = 0b%b, red_op_B = 0b%b,
bypass_A = 0b%b, bypass_B = 0b%b, serial_in = 0b%b, direction = %s, opcode = %s, A =
0b%b, B = 0b%b",
                      super.convert2string(), rst, cin, red_op_A, red_op_B, bypass_A,
bypass_B, serial_in, direction, opcode, A, B);
  endfunction


  // Reset distribution
  constraint c_reset {
    rst dist {1 := 5, 0 := 95};
  }

  // Arithmetic operations
  constraint C_arth {
    if (opcode == ADD || opcode == MULT) {
      A dist {MAXPOS := 40, ZERO := 40, MAXNEG := 40, other := 20};
      B dist {MAXPOS := 40, ZERO := 40, MAXNEG := 40, other := 20};
    }
  }

  // Reduction operations
  constraint C_read_A_B {
    if (opcode == OR || opcode == XOR) {
      if (red_op_A) {
        A dist {0 := 20, one_bit_high := 80};
        B == 0;
      }
      else if (red_op_B) {
        B dist {0 := 20, one_bit_high := 80};
        A == 0;
      }
    }
  }
```

```
  // Opcode distribution
  constraint c_opcode {
    opcode dist {
      OR := 15, XOR := 15,
      ADD := 20, MULT := 20,
      SHIFT := 10, ROTATE := 10,
      INVALID_6 := 5, INVALID_7 := 5
    };
  }

  // Bypass behavior
  constraint c_bypass {
    bypass_A dist {1 := 10, 0 := 90};
    bypass_B dist {1 := 10, 0 := 90};
  }

  // Invalid reduction usage
  constraint c_invalid_red_op_A_B {
    if (opcode != OR && opcode != XOR) {
      red_op_A dist {0 := 80, 1 := 20};
      red_op_B dist {0 := 80, 1 := 20};
    }
  }

  // Unique opcode array
  constraint unique_opcode {
    foreach (arr_opcode[i]) {
      arr_opcode[i] inside {[OR:ROTATE]};
      foreach (arr_opcode[j]) {
        if (i != j)
          arr_opcode[i] != arr_opcode[j];
      }
    }
  }

endclass

endpackage
```

sequencer :

```
package ALSU_sequencer_pkg;
import ALSU_seq_item_pkg ::*;
import uvm_pkg::*;
`include "uvm_macros.svh"
class ALSU_sequencer extends uvm_sequencer #(ALSU_seq_item);
  `uvm_component_utils(ALSU_sequencer);

  function new (string name = "ALSU_sequencer", uvm_component parent = null);
    super.new(name, parent);
  endfunction
```

```
endclass
endpackage
```

assertions :

```
module ALSU_sva (ALSU_if alsu);


  property p_reset;
    @(posedge alsu.clk)
    alsu.rst |=> (alsu.out == 0 && alsu.leds == 0);
  endproperty

  a_reset: assert property (p_reset);



  property p_add;
    @(posedge alsu.clk)
    (alsu.opcode == 3'h2 && !alsu.rst)
    |=> alsu.out == (alsu.A + alsu.B + alsu.cin);
  endproperty

  a_add: assert property (p_add);



  property p_mult;
    @(posedge alsu.clk)
    (alsu.opcode == 3'h3 && !alsu.rst)
    |=> alsu.out == (alsu.A * alsu.B);
  endproperty

  a_mult: assert property (p_mult);



  property p_or;
    @(posedge alsu.clk)
    (alsu.opcode == 3'h0 && !alsu.red_op_A && !alsu.red_op_B)
    |=> alsu.out == (alsu.A | alsu.B);
  endproperty

  a_or: assert property (p_or);


  property p_xor;
    @(posedge alsu.clk)
    (alsu.opcode == 3'h1 && !alsu.red_op_A && !alsu.red_op_B)
    |=> alsu.out == (alsu.A ^ alsu.B);
  endproperty
```

```
  a_xor: assert property (p_xor);



  property p_bypass_A;
    @(posedge alsu.clk)
    (alsu.bypass_A && !alsu.rst)
    |=> alsu.out == alsu.A;
  endproperty

  a_bypass_A: assert property (p_bypass_A);



  property p_bypass_B;
    @(posedge alsu.clk)
    (alsu.bypass_B && !alsu.rst && !alsu.bypass_A)
    |=> alsu.out == alsu.B;
  endproperty

  a_bypass_B: assert property (p_bypass_B);


  property p_invalid_out;
    @(posedge alsu.clk)
    ((alsu.opcode[1] & alsu.opcode[2]) ||
     ((alsu.red_op_A | alsu.red_op_B) && (alsu.opcode[1] | alsu.opcode[2])))
    |=> alsu.out == 0;
  endproperty

  a_invalid_out: assert property (p_invalid_out);


  property p_invalid_leds;
    @(posedge alsu.clk)
    ((alsu.opcode[1] & alsu.opcode[2]) ||
     ((alsu.red_op_A | alsu.red_op_B) && (alsu.opcode[1] | alsu.opcode[2])))
    |=> alsu.leds == ~$past(alsu.leds);
  endproperty

  a_invalid_leds: assert property (p_invalid_leds);



  c_add:     cover property (@(posedge alsu.clk) alsu.opcode == 3'h2);
  c_mult:    cover property (@(posedge alsu.clk) alsu.opcode == 3'h3);
  c_shift:   cover property (@(posedge alsu.clk) alsu.opcode == 3'h4);
  c_rotate:  cover property (@(posedge alsu.clk) alsu.opcode == 3'h5);
  c_invalid: cover property (@(posedge alsu.clk) (alsu.opcode[1] & alsu.opcode[2]));

endmodule
```

test :

```
package ALSU_test_pkg;
import shift_reg_cfg_pkg::*;
import ALSU_cfg_pkg::*;
import ALSU_env_pkg::*;
import shift_reg_agent_pkg::*;
import uvm_pkg::*;
import ALSU_main_sequence_pkg::*;
`include "uvm_macros.svh"

class ALSU_test extends uvm_test;

  `uvm_component_utils(ALSU_test)

  ALSU_env env;

  ALSU_config  ALSU_cfg;
  shift_reg_config shift_cfg;


  virtual ALSU_if ALSU_vif;
  virtual shift_reg_if shift_vif;


  ALSU_main_sequence main_seq;
  ALSU_reset_sequence reset_seq;

  function new(string name = "ALSU_env", uvm_component parent = null);
    super.new(name,parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);


    env = ALSU_env::type_id::create("env", this);


    ALSU_cfg  = ALSU_config::type_id::create("ALSU_cfg");
    shift_cfg = shift_reg_config::type_id::create("shift_cfg");


    main_seq  = ALSU_main_sequence::type_id::create("main_seq", this);
    reset_seq = ALSU_reset_sequence::type_id::create("reset_seq", this);


    if(!uvm_config_db #(virtual ALSU_if)::get(this, "", "ALSU_if", ALSU_cfg.ALSU_vif))
      `uvm_fatal("build_phase", "Cannot get ALSU_if")

    if(!uvm_config_db #(virtual shift_reg_if)::get(this, "", "shift_reg_if", shift_vif))
      `uvm_fatal("build_phase", "Cannot get shift_reg_if")
```

```
    shift_cfg.shift_reg_vif = shift_vif;
    shift_cfg.is_active = UVM_PASSIVE;


    uvm_config_db #(ALSU_config)::set(this, "*", "CFG", ALSU_cfg);
    uvm_config_db #(shift_reg_config)::set(this, "*", "SHIFT_CFG", shift_cfg);

  endfunction


  task run_phase(uvm_phase phase);
    super.run_phase(phase);

    phase.raise_objection(this);

    `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
    reset_seq.start(env.agt.sqr);

    `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
    main_seq.start(env.agt.sqr);

    `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)

    phase.drop_objection(this);
  endtask

endclass

endpackage
```

list :

src_files.list

```
shared_package_ALSU.sv
shared_package_shift_reg.sv
ALSU_Seq_item.sv
ALSU_if.sv
shift_reg_if.sv
ALSU_design.sv
shift_reg.v
ALSU_configs.sv
shift_reg_configs.sv
ALSU_driver.sv
shift_reg_driver.sv
ALSU_monitor.sv
shift_reg_monitor.sv
ALSU_sequencer.sv
shift_reg_sequencer.sv
ALSU_agent.sv
shift_reg_agent.sv
ALSU_scoreboard.sv
ALSU_coverage_collector.sv
ALSU_env.sv
ALSU_main_seq.sv
ALSU_test.sv
top.sv
```

Do file :

run.do

```
vlib work
vlog -f src_files.list
vsim -voptargs=+acc work.top -classdebug -uvmcontrol=all
add wave /top/alsu/*
run -all
```

Transcript showing zero errors and zero fatal errors :

```
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    7
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [run_phase]     3
# ** Note: $finish    : C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 20002 ns  Iteration: 61  Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430

VSIM 2>
```

Assertions all passed :

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Cou
/uvm_pkg::uvm_reg_map::do_write/#ublk#215181159#1731/imm...	Immediate	SVA	on	0	0	
/uvm_pkg::uvm_reg_map::do_read/#ublk#215181159#1771/imm...	Immediate	SVA	on	0	0	
/ALSU_main_sequence_pkg::ALSU_main_sequence::body/#ublk#...	Immediate	SVA	on	0	0	
/top/t1/sva_inst/a_reset	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_add	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_mult	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_or	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_xor	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_bypass_A	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_bypass_B	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_invalid_out	Concurrent	SVA	on	0	1	
/top/t1/sva_inst/a_invalid_leds	Concurrent	SVA	on	0	1	